

SQOOP & HIVE LAB ASSIGNMENTS

STARTING / COMMON SETUP

1. Start Hadoop Distributed File System (DFS) and YARN

```
start-dfs.sh  
start-yarn.sh
```

2. Verify Hadoop services

```
jps  
hdfs dfs -ls /  
yarn node -list
```

3. Start MySQL

```
sudo service mysql start  
# If your system uses systemd:  
sudo systemctl start mysql
```

4. Open MySQL

```
mysql -u root -p
```

5. Create the Sqoop user and grant privileges

```
CREATE USER IF NOT EXISTS 'msis'@'localhost' IDENTIFIED BY 'mahe';  
GRANT ALL PRIVILEGES ON *.* TO 'msis'@'localhost';  
FLUSH PRIVILEGES;
```

6. Test Sqoop access

```
exit  
sqoop list-databases \  
--connect jdbc:mysql://localhost?useSSL=false \  
--username msis --password mahe
```

Used Terminal 1 for MySQL/SQL and Terminal 2 for Hadoop, HDFS, Sqoop and Hive.

ASSIGNMENT 1 — E-COMMERCE SALES ANALYTICS

Scenario: An e-commerce company stores transactional data in MySQL and wants to move historical sales data to Hadoop for analytics.

MySQL tables: customers(customer_id, name, city, state, join_date), products(product_id, product_name, category, price), and orders(order_id, customer_id, product_id, quantity, order_date, order_status).

Q1. Create ecommerce_db and all three tables. Insert at least 30 customers, 15 products and 100 orders.

```
mysql -u msis -p
DROP DATABASE IF EXISTS ecommerce_db;
CREATE DATABASE ecommerce_db;
USE ecommerce_db;

CREATE TABLE customers(
customer_id INT PRIMARY KEY,name VARCHAR(100),city VARCHAR(50),
state VARCHAR(50),join_date DATE);

INSERT INTO customers VALUES
(1,'Rahul','Bengaluru','Karnataka','2023-01-10'),
(2,'Priya','Mysuru','Karnataka','2023-01-15'),
(3,'Arjun','Mangaluru','Karnataka','2023-02-05'),
(4,'Sneha','Chennai','Tamil Nadu','2023-02-20'),
(5,'Kiran','Hyderabad','Telangana','2023-03-01'),
(6,'Ananya','Pune','Maharashtra','2023-03-12'),
(7,'Rohit','Mumbai','Maharashtra','2023-03-25'),
(8,'Neha','Delhi','Delhi','2023-04-05'),
(9,'Vikram','Kolkata','West Bengal','2023-04-18'),
(10,'Asha','Bengaluru','Karnataka','2023-05-01'),
(11,'Manoj','Mysuru','Karnataka','2023-05-10'),
(12,'Divya','Chennai','Tamil Nadu','2023-05-20'),
(13,'Suresh','Hyderabad','Telangana','2023-06-02'),
(14,'Pooja','Pune','Maharashtra','2023-06-15'),
(15,'Akash','Mumbai','Maharashtra','2023-06-25'),
(16,'Meera','Delhi','Delhi','2023-07-05'),
(17,'Nikhil','Kolkata','West Bengal','2023-07-18'),
(18,'Kavya','Bengaluru','Karnataka','2023-08-01'),
(19,'Aditya','Mysuru','Karnataka','2023-08-10'),
(20,'Riya','Chennai','Tamil Nadu','2023-08-22'),
(21,'Varun','Hyderabad','Telangana','2023-09-05'),
(22,'Isha','Pune','Maharashtra','2023-09-15'),
(23,'Harish','Mumbai','Maharashtra','2023-09-25'),
(24,'Swathi','Delhi','Delhi','2023-10-05'),
(25,'Karthik','Kolkata','West Bengal','2023-10-15'),
(26,'Nandini','Bengaluru','Karnataka','2023-10-25'),
(27,'Yash','Mysuru','Karnataka','2023-11-05'),
(28,'Deepa','Chennai','Tamil Nadu','2023-11-15'),
(29,'Ajay','Hyderabad','Telangana','2023-12-01'),
(30,'Lakshmi','Pune','Maharashtra','2023-12-15');

CREATE TABLE products(
product_id INT PRIMARY KEY,product_name VARCHAR(100),
category VARCHAR(50),price DECIMAL(10,2));

INSERT INTO products VALUES
```

```
(1,'Laptop','Electronics',55000),(2,'Mobile Phone','Electronics',25000),
(3,'Headphones','Electronics',2000),(4,'Keyboard','Electronics',1200),
(5,'Mouse','Electronics',800),(6,'T-Shirt','Fashion',900),
(7,'Jeans','Fashion',1800),(8,'Shoes','Fashion',2500),
(9,'Jacket','Fashion',3000),(10,'Backpack','Fashion',1500),
(11,'Book','Books',500),(12,'Notebook','Books',200),
(13,'Pen Set','Books',150),(14,'Coffee Maker','Home',4500),
(15,'Table Lamp','Home',1800);
```

```
CREATE TABLE orders(
order_id INT PRIMARY KEY,customer_id INT,product_id INT,
quantity INT,order_date DATE,order_status VARCHAR(20),
FOREIGN KEY(customer_id) REFERENCES customers(customer_id),
FOREIGN KEY(product_id) REFERENCES products(product_id));
```

```
DROP PROCEDURE IF EXISTS generate_orders;
DELIMITER //
CREATE PROCEDURE generate_orders()
BEGIN
DECLARE i INT DEFAULT 1;
WHILE i<=100 DO
INSERT INTO orders VALUES(i,MOD(i-1,30)+1,MOD(i-1,15)+1,
MOD(i,5)+1,DATE_ADD('2024-01-01',INTERVAL MOD(i-1,365) DAY),
CASE WHEN MOD(i,10)=0 THEN 'Cancelled'
WHEN MOD(i,7)=0 THEN 'Pending' ELSE 'Completed' END);
SET i=i+1;
END WHILE;
END //
DELIMITER ;
CALL generate_orders();
```

```
SELECT COUNT(*) FROM customers;
SELECT COUNT(*) FROM products;
SELECT COUNT(*) FROM orders;
```

Q2. Use Sqoop to import all three tables into separate directories under /me/ecommerce/.

```
hdfs dfs -rm -r -f /me/ecommerce
hdfs dfs -mkdir -p /me/ecommerce
```

```
sqoop import --connect jdbc:mysql://localhost:3306/ecommerce_db?useSSL=false --username msis --
password mahe --table customers --target-dir /me/ecommerce/customers --num-mappers 1
```

```
sqoop import --connect jdbc:mysql://localhost:3306/ecommerce_db?useSSL=false --username msis --
password mahe --table products --target-dir /me/ecommerce/products --num-mappers 1
```

```
sqoop import --connect jdbc:mysql://localhost:3306/ecommerce_db?useSSL=false --username msis --
password mahe --table orders --target-dir /me/ecommerce/orders --num-mappers 1
```

Q3. Verify imported data using HDFS commands and report files/records.

```
hdfs dfs -ls /me/ecommerce
hdfs dfs -ls /me/ecommerce/customers
hdfs dfs -ls /me/ecommerce/products
hdfs dfs -ls /me/ecommerce/orders
hdfs dfs -cat /me/ecommerce/customers/part-* | wc -l
hdfs dfs -cat /me/ecommerce/products/part-* | wc -l
hdfs dfs -cat /me/ecommerce/orders/part-* | wc -l
```

Expected records: 30 customers, 15 products and 100 orders.

Q4. Create Hive database ecommerce_dw and suitable tables.

```
hive
CREATE DATABASE IF NOT EXISTS ecommerce_dw;
USE ecommerce_dw;

CREATE EXTERNAL TABLE customers(
customer_id INT,name STRING,city STRING,state STRING,join_date DATE)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ';'
STORED AS TEXTFILE LOCATION '/me/ecommerce/customers';

CREATE EXTERNAL TABLE products(
product_id INT,product_name STRING,category STRING,price DECIMAL(10,2))
ROW FORMAT DELIMITED FIELDS TERMINATED BY ';'
STORED AS TEXTFILE LOCATION '/me/ecommerce/products';

CREATE EXTERNAL TABLE orders(
order_id INT,customer_id INT,product_id INT,quantity INT,
order_date DATE,order_status STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ';'
STORED AS TEXTFILE LOCATION '/me/ecommerce/orders';

SHOW TABLES;
```

Q5. Write HiveQL queries.

Q5.1 Total sales revenue

```
SELECT SUM(o.quantity*p.price) AS total_sales_revenue FROM orders o JOIN products p ON
o.product_id=p.product_id WHERE o.order_status='Completed';
```

Q5.2 Revenue by product category

```
SELECT p.category,SUM(o.quantity*p.price) AS revenue FROM orders o JOIN products p ON
o.product_id=p.product_id WHERE o.order_status='Completed' GROUP BY p.category ORDER BY
revenue DESC;
```

Q5.3 Top 5 customers by spending

```
SELECT c.customer_id,c.name,SUM(o.quantity*p.price) total_spending FROM orders o JOIN customers
c ON o.customer_id=c.customer_id JOIN products p ON o.product_id=p.product_id WHERE
```

```
o.order_status='Completed' GROUP BY c.customer_id,c.name ORDER BY total_spending DESC LIMIT 5;
```

Q5.4 Top 5 products by quantity sold

```
SELECT p.product_id,p.product_name,SUM(o.quantity) total_quantity FROM orders o JOIN products p ON o.product_id=p.product_id WHERE o.order_status='Completed' GROUP BY p.product_id,p.product_name ORDER BY total_quantity DESC LIMIT 5;
```

Q5.5 Monthly sales revenue

```
SELECT YEAR(o.order_date) year,MONTH(o.order_date) month,SUM(o.quantity*p.price) monthly_revenue FROM orders o JOIN products p ON o.product_id=p.product_id WHERE o.order_status='Completed' GROUP BY YEAR(o.order_date),MONTH(o.order_date) ORDER BY year,month;
```

Q5.6 City with the highest sales

```
SELECT c.city,SUM(o.quantity*p.price) total_sales FROM orders o JOIN customers c ON o.customer_id=c.customer_id JOIN products p ON o.product_id=p.product_id WHERE o.order_status='Completed' GROUP BY c.city ORDER BY total_sales DESC LIMIT 1;
```

Q5.7 Percentage of cancelled orders

```
SELECT ROUND(100.0*SUM(CASE WHEN order_status='Cancelled' THEN 1 ELSE 0 END)/COUNT(*),2) cancelled_percentage FROM orders;
```

Q6. Store each individual query result in a separate HDFS file.

```
exit
hdfs dfs -mkdir -p /me/ecommerce/results
```

```
hive -S -e "USE ecommerce_dw; SELECT SUM(o.quantity*p.price) FROM orders o JOIN products p ON o.product_id=p.product_id WHERE o.order_status='Completed';" > /tmp/total_sales_revenue.txt
hdfs dfs -put -f /tmp/total_sales_revenue.txt /me/ecommerce/results/
```

```
hive -S -e "USE ecommerce_dw; SELECT p.category,SUM(o.quantity*p.price) FROM orders o JOIN products p ON o.product_id=p.product_id WHERE o.order_status='Completed' GROUP BY p.category;" > /tmp/revenue_by_category.txt
hdfs dfs -put -f /tmp/revenue_by_category.txt /me/ecommerce/results/
```

```
hive -S -e "USE ecommerce_dw; SELECT c.customer_id,c.name,SUM(o.quantity*p.price) FROM orders o JOIN customers c ON o.customer_id=c.customer_id JOIN products p ON o.product_id=p.product_id WHERE o.order_status='Completed' GROUP BY c.customer_id,c.name ORDER BY SUM(o.quantity*p.price) DESC LIMIT 5;" > /tmp/top5_customers.txt
hdfs dfs -put -f /tmp/top5_customers.txt /me/ecommerce/results/
```

```
hive -S -e "USE ecommerce_dw; SELECT p.product_id,p.product_name,SUM(o.quantity) FROM orders o JOIN products p ON o.product_id=p.product_id WHERE o.order_status='Completed' GROUP BY p.product_id,p.product_name ORDER BY SUM(o.quantity) DESC LIMIT 5;" > /tmp/top5_products.txt
hdfs dfs -put -f /tmp/top5_products.txt /me/ecommerce/results/
```

```
hive -S -e "USE ecommerce_dw; SELECT
YEAR(o.order_date),MONTH(o.order_date),SUM(o.quantity*p.price) FROM orders o JOIN products p ON
o.product_id=p.product_id WHERE o.order_status='Completed' GROUP BY
YEAR(o.order_date),MONTH(o.order_date);" > /tmp/monthly_sales.txt
hdfs dfs -put -f /tmp/monthly_sales.txt /me/ecommerce/results/

hive -S -e "USE ecommerce_dw; SELECT c.city,SUM(o.quantity*p.price) FROM orders o JOIN customers c
ON o.customer_id=c.customer_id JOIN products p ON o.product_id=p.product_id WHERE
o.order_status='Completed' GROUP BY c.city ORDER BY SUM(o.quantity*p.price) DESC LIMIT 1;" >
/tmp/top_city.txt
hdfs dfs -put -f /tmp/top_city.txt /me/ecommerce/results/

hive -S -e "USE ecommerce_dw; SELECT ROUND(100.0*SUM(CASE WHEN order_status='Cancelled'
THEN 1 ELSE 0 END)/COUNT(*),2) FROM orders;" > /tmp/cancelled_percentage.txt
hdfs dfs -put -f /tmp/cancelled_percentage.txt /me/ecommerce/results/
```

```
hdfs dfs -ls /me/ecommerce/results
```

Q7. Transfer result files back to local system.

```
mkdir -p ~/ecommerce_results
hdfs dfs -get /me/ecommerce/results/* ~/ecommerce_results/
ls ~/ecommerce_results
```

ASSIGNMENT 2 — UNIVERSITY STUDENT PERFORMANCE ANALYTICS

Scenario: A university maintains student, course, enrollment, and examination information in MySQL. The university wants to analyze academic performance using Hadoop.

MySQL tables: students(student_id, student_name, department, admission_year, gender), courses(course_id, course_name, department, credits), and results(result_id, student_id, course_id, semester, marks, attendance).

Q1. Create university_db and populate at least 40 students, 10 courses and 150 results.

```
mysql -u msis -p
DROP DATABASE IF EXISTS university_db;
CREATE DATABASE university_db;
USE university_db;

CREATE TABLE students(student_id INT PRIMARY KEY,student_name VARCHAR(100),
department VARCHAR(50),admission_year INT,gender VARCHAR(10));

INSERT INTO students VALUES
(1,'Student1','CSE',2022,'Male'),(2,'Student2','CSE',2022,'Female'),
(3,'Student3','CSE',2023,'Male'),(4,'Student4','CSE',2023,'Female'),
(5,'Student5','ISE',2022,'Male'),(6,'Student6','ISE',2022,'Female'),
(7,'Student7','ISE',2023,'Male'),(8,'Student8','ISE',2023,'Female'),
(9,'Student9','ECE',2022,'Male'),(10,'Student10','ECE',2022,'Female'),
(11,'Student11','ECE',2023,'Male'),(12,'Student12','ECE',2023,'Female'),
```

```
(13,'Student13','ME',2022,'Male'),(14,'Student14','ME',2022,'Female'),
(15,'Student15','ME',2023,'Male'),(16,'Student16','ME',2023,'Female'),
(17,'Student17','CSE',2022,'Male'),(18,'Student18','CSE',2022,'Female'),
(19,'Student19','ISE',2023,'Male'),(20,'Student20','ISE',2023,'Female'),
(21,'Student21','ECE',2022,'Male'),(22,'Student22','ECE',2022,'Female'),
(23,'Student23','ME',2023,'Male'),(24,'Student24','ME',2023,'Female'),
(25,'Student25','CSE',2022,'Male'),(26,'Student26','CSE',2023,'Female'),
(27,'Student27','ISE',2022,'Male'),(28,'Student28','ISE',2023,'Female'),
(29,'Student29','ECE',2022,'Male'),(30,'Student30','ECE',2023,'Female'),
(31,'Student31','ME',2022,'Male'),(32,'Student32','ME',2023,'Female'),
(33,'Student33','CSE',2022,'Male'),(34,'Student34','CSE',2023,'Female'),
(35,'Student35','ISE',2022,'Male'),(36,'Student36','ISE',2023,'Female'),
(37,'Student37','ECE',2022,'Male'),(38,'Student38','ECE',2023,'Female'),
(39,'Student39','ME',2022,'Male'),(40,'Student40','ME',2023,'Female');
```

```
CREATE TABLE courses(course_id INT PRIMARY KEY,course_name VARCHAR(100),
department VARCHAR(50),credits INT);
```

```
INSERT INTO courses VALUES
```

```
(1,'Data Structures','CSE',4),(2,'DBMS','CSE',4),
(3,'Operating Systems','CSE',4),(4,'Computer Networks','ISE',4),
(5,'Software Engineering','ISE',3),(6,'Digital Electronics','ECE',4),
(7,'Microprocessors','ECE',4),(8,'Thermodynamics','ME',4),
(9,'Machine Design','ME',4),(10,'Engineering Mathematics','Common',4);
```

```
CREATE TABLE results(result_id INT PRIMARY KEY,student_id INT,course_id INT,
semester INT,marks DECIMAL(5,2),attendance DECIMAL(5,2),
FOREIGN KEY(student_id) REFERENCES students(student_id),
FOREIGN KEY(course_id) REFERENCES courses(course_id));
```

```
DROP PROCEDURE IF EXISTS generate_results;
```

```
DELIMITER //
```

```
CREATE PROCEDURE generate_results()
```

```
BEGIN
```

```
DECLARE i INT DEFAULT 1;
```

```
WHILE i<=150 DO
```

```
INSERT INTO results VALUES(i,MOD(i-1,40)+1,MOD(i-1,10)+1,
MOD(i-1,8)+1,45+MOD(i*7,51),60+MOD(i*11,41));
```

```
SET i=i+1;
```

```
END WHILE;
```

```
END //
```

```
DELIMITER ;
```

```
CALL generate_results();
```

```
SELECT COUNT(*) FROM students; SELECT COUNT(*) FROM courses; SELECT COUNT(*) FROM
results;
```

Q2. Import the tables using Sqoop into /me/university/.

```
hdfs dfs -rm -r -f /me/university
```

```
hdfs dfs -mkdir -p /me/university
```

```
sqoop import --connect jdbc:mysql://localhost:3306/university_db?useSSL=false --username msis --
```

```
password mahe --table students --target-dir /me/university/students --num-mappers 1
sqoop import --connect jdbc:mysql://localhost:3306/university_db?useSSL=false --username msis --
password mahe --table courses --target-dir /me/university/courses --num-mappers 1
```

Q3. Import results using multiple Sqoop mappers and explain --split-by.

```
sqoop import --connect jdbc:mysql://localhost:3306/university_db?useSSL=false --username msis --
password mahe --table results --target-dir /me/university/results --num-mappers 4 --split-by result_id
```

--split-by specifies the column Sqoop uses to divide source rows into ranges for parallel mapper imports. result_id is numeric and unique, so it is appropriate. Four mappers can import different ranges in parallel.

Q4. Inspect the resulting HDFS directories and files.

```
hdfs dfs -ls /me/university
hdfs dfs -ls /me/university/results
hdfs dfs -cat /me/university/students/part-* | wc -l
hdfs dfs -cat /me/university/courses/part-* | wc -l
hdfs dfs -cat /me/university/results/part-* | wc -l
```

Q5. Create appropriate Hive tables.

```
hive
CREATE DATABASE IF NOT EXISTS university_dw;
USE university_dw;
CREATE EXTERNAL TABLE students(student_id INT,student_name STRING,department
STRING,admission_year INT,gender STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION
'/me/university/students';
CREATE EXTERNAL TABLE courses(course_id INT,course_name STRING,department
STRING,credits INT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION
'/me/university/courses';
CREATE EXTERNAL TABLE results(result_id INT,student_id INT,course_id INT,semester INT,marks
DECIMAL(5,2),attendance DECIMAL(5,2))
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION
'/me/university/results';
```

Q6. Write Hive queries.

Q6.1 Average marks for each department

```
SELECT s.department,AVG(r.marks) average_marks FROM students s JOIN results r ON
s.student_id=r.student_id GROUP BY s.department;
```

Q6.2 Top 3 students in each department

```
SELECT department,student_name,average_marks FROM (SELECT
s.department,s.student_name,AVG(r.marks) average_marks,ROW_NUMBER() OVER(PARTITION BY
s.department ORDER BY AVG(r.marks) DESC) rank_no FROM students s JOIN results r ON
s.student_id=r.student_id GROUP BY s.department,s.student_name) x WHERE rank_no<=3;
```

Q6.3 Pass percentage for each course

```
SELECT c.course_id,c.course_name,ROUND(100.0*SUM(CASE WHEN r.marks>=40 THEN 1 ELSE 0
END)/COUNT(*),2) pass_percentage FROM courses c JOIN results r ON c.course_id=r.course_id
GROUP BY c.course_id,c.course_name;
```

Q6.4 Students having attendance below 75%

```
SELECT s.student_id,s.student_name,AVG(r.attendance) average_attendance FROM students s JOIN
results r ON s.student_id=r.student_id GROUP BY s.student_id,s.student_name HAVING
AVG(r.attendance)<75;
```

Q6.5 Relationship between attendance and average marks

```
SELECT CASE WHEN attendance<75 THEN 'Below 75' WHEN attendance<85 THEN '75-84' WHEN
attendance<95 THEN '85-94' ELSE '95+' END attendance_range,AVG(marks) average_marks FROM
results GROUP BY CASE WHEN attendance<75 THEN 'Below 75' WHEN attendance<85 THEN '75-84'
WHEN attendance<95 THEN '85-94' ELSE '95+' END;
```

Q6.6 Course having the highest failure rate

```
SELECT c.course_name,ROUND(100.0*SUM(CASE WHEN r.marks<40 THEN 1 ELSE 0
END)/COUNT(*),2) failure_rate FROM courses c JOIN results r ON c.course_id=r.course_id GROUP
BY c.course_name ORDER BY failure_rate DESC LIMIT 1;
```

Q6.7 Semester-wise average performance

```
SELECT semester,AVG(marks) average_marks FROM results GROUP BY semester ORDER BY
semester;
```

Q7. Store each individual query result into a separate HDFS file.

```
exit
hdfs dfs -mkdir -p /me/university/results_analysis
```

```
hive -S -e "USE university_dw; SELECT s.department,AVG(r.marks) FROM students s JOIN results r ON
s.student_id=r.student_id GROUP BY s.department;" > /tmp/avg_marks_department.txt
hdfs dfs -put -f /tmp/avg_marks_department.txt /me/university/results_analysis/
```

```
hive -S -e "USE university_dw; SELECT department,student_name,average_marks FROM (SELECT
s.department,s.student_name,AVG(r.marks) average_marks,ROW_NUMBER() OVER(PARTITION BY
s.department ORDER BY AVG(r.marks) DESC) rank_no FROM students s JOIN results r ON
s.student_id=r.student_id GROUP BY s.department,s.student_name) x WHERE rank_no<=3;" >
/tmp/top3_students.txt
hdfs dfs -put -f /tmp/top3_students.txt /me/university/results_analysis/
```

```
hive -S -e "USE university_dw; SELECT c.course_name,ROUND(100.0*SUM(CASE WHEN r.marks>=40
THEN 1 ELSE 0 END)/COUNT(*),2) FROM courses c JOIN results r ON c.course_id=r.course_id GROUP
BY c.course_name;" > /tmp/pass_percentage_course.txt
hdfs dfs -put -f /tmp/pass_percentage_course.txt /me/university/results_analysis/
```

```
hive -S -e "USE university_dw; SELECT s.student_id,s.student_name,AVG(r.attendance) FROM students s
JOIN results r ON s.student_id=r.student_id GROUP BY s.student_id,s.student_name HAVING
```

```

AVG(r.attendance)<75;" > /tmp/attendance_below_75.txt
hdfs dfs -put -f /tmp/attendance_below_75.txt /me/university/results_analysis/

hive -S -e "USE university_dw; SELECT CASE WHEN attendance<75 THEN 'Below 75' WHEN
attendance<85 THEN '75-84' WHEN attendance<95 THEN '85-94' ELSE '95+' END,AVG(marks) FROM
results GROUP BY CASE WHEN attendance<75 THEN 'Below 75' WHEN attendance<85 THEN '75-84'
WHEN attendance<95 THEN '85-94' ELSE '95+' END;" > /tmp/attendance_marks_relationship.txt
hdfs dfs -put -f /tmp/attendance_marks_relationship.txt /me/university/results_analysis/

hive -S -e "USE university_dw; SELECT c.course_name,100.0*SUM(CASE WHEN r.marks<40 THEN 1
ELSE 0 END)/COUNT(*) FROM courses c JOIN results r ON c.course_id=r.course_id GROUP BY
c.course_name ORDER BY 100.0*SUM(CASE WHEN r.marks<40 THEN 1 ELSE 0 END)/COUNT(*) DESC
LIMIT 1;" > /tmp/highest_failure_course.txt
hdfs dfs -put -f /tmp/highest_failure_course.txt /me/university/results_analysis/

hive -S -e "USE university_dw; SELECT semester,AVG(marks) FROM results GROUP BY semester ORDER
BY semester;" > /tmp/semester_average.txt
hdfs dfs -put -f /tmp/semester_average.txt /me/university/results_analysis/

hdfs dfs -ls /me/university/results_analysis

```

Q8. Transfer result files back to local system.

```

mkdir -p ~/university_results
hdfs dfs -get /me/university/results_analysis/* ~/university_results/
ls ~/university_results

```

ASSIGNMENT 3 — BANKING TRANSACTION AND CUSTOMER ANALYTICS

Scenario: A bank stores customer accounts and transactions in MySQL. Its analytics team needs a Hadoop-based solution for transaction analysis.

MySQL tables: customers(customer_id, customer_name, city, account_type, account_open_date), accounts(account_id, customer_id, balance, branch), and transactions(transaction_id, account_id, transaction_type, amount, transaction_date).

Populate the database with at least 30 customers, 40 accounts, and 200 transactions

Q1. Create and populate bank_db with at least 30 customers, 40 accounts and 200 transactions.

```

mysql -u msis -p
DROP DATABASE IF EXISTS bank_db;
CREATE DATABASE bank_db;
USE bank_db;

CREATE TABLE customers(customer_id INT PRIMARY KEY,customer_name VARCHAR(100),city
VARCHAR(50),account_type VARCHAR(30),account_open_date DATE);

```

```
INSERT INTO customers VALUES
```

```
(1,'Customer1','Bengaluru','Savings','2022-01-01'),(2,'Customer2','Mysuru','Current','2022-01-05'),
(3,'Customer3','Chennai','Savings','2022-01-10'),(4,'Customer4','Hyderabad','Current','2022-01-15'),
(5,'Customer5','Pune','Savings','2022-02-01'),(6,'Customer6','Mumbai','Current','2022-02-05'),
(7,'Customer7','Delhi','Savings','2022-02-10'),(8,'Customer8','Kolkata','Current','2022-02-15'),
(9,'Customer9','Bengaluru','Savings','2022-03-01'),(10,'Customer10','Mysuru','Current','2022-03-05'),
(11,'Customer11','Chennai','Savings','2022-03-10'),(12,'Customer12','Hyderabad','Current','2022-03-15'),
(13,'Customer13','Pune','Savings','2022-04-01'),(14,'Customer14','Mumbai','Current','2022-04-05'),
(15,'Customer15','Delhi','Savings','2022-04-10'),(16,'Customer16','Kolkata','Current','2022-04-15'),
(17,'Customer17','Bengaluru','Savings','2022-05-01'),(18,'Customer18','Mysuru','Current','2022-05-05'),
(19,'Customer19','Chennai','Savings','2022-05-10'),(20,'Customer20','Hyderabad','Current','2022-05-15'),
(21,'Customer21','Pune','Savings','2022-06-01'),(22,'Customer22','Mumbai','Current','2022-06-05'),
(23,'Customer23','Delhi','Savings','2022-06-10'),(24,'Customer24','Kolkata','Current','2022-06-15'),
(25,'Customer25','Bengaluru','Savings','2022-07-01'),(26,'Customer26','Mysuru','Current','2022-07-05'),
(27,'Customer27','Chennai','Savings','2022-07-10'),(28,'Customer28','Hyderabad','Current','2022-07-15'),
(29,'Customer29','Pune','Savings','2022-08-01'),(30,'Customer30','Mumbai','Current','2022-08-05');
```

```
CREATE TABLE accounts(account_id INT PRIMARY KEY,customer_id INT,balance
DECIMAL(12,2),branch VARCHAR(50),FOREIGN KEY(customer_id) REFERENCES
customers(customer_id));
```

```
DROP PROCEDURE IF EXISTS generate_accounts;
```

```
DELIMITER //
```

```
CREATE PROCEDURE generate_accounts()
```

```
BEGIN
```

```
DECLARE i INT DEFAULT 1;
```

```
WHILE i<=40 DO
```

```
INSERT INTO accounts VALUES(i,MOD(i-1,30)+1,5000+MOD(i*1234,50000),
```

```
CASE MOD(i,5) WHEN 0 THEN 'Bengaluru' WHEN 1 THEN 'Mysuru' WHEN 2 THEN 'Chennai'
WHEN 3 THEN 'Hyderabad' ELSE 'Pune' END);
```

```
SET i=i+1;
```

```
END WHILE;
```

```
END //
```

```
DELIMITER ;
```

```
CALL generate_accounts();
```

```
CREATE TABLE transactions(transaction_id INT PRIMARY KEY,account_id INT,transaction_type
VARCHAR(20),amount DECIMAL(12,2),transaction_date DATE,FOREIGN KEY(account_id)
REFERENCES accounts(account_id));
```

```
DROP PROCEDURE IF EXISTS generate_transactions;
```

```
DELIMITER //
```

```
CREATE PROCEDURE generate_transactions()
```

```
BEGIN
```

```
DECLARE i INT DEFAULT 1;
```

```
WHILE i<=200 DO
```

```
INSERT INTO transactions VALUES(i,MOD(i-1,40)+1,CASE WHEN MOD(i,2)=0 THEN 'Deposit'
ELSE 'Withdrawal' END,1000+MOD(i*777,20000),DATE_ADD('2024-01-01',INTERVAL MOD(i-
1,365) DAY));
```

```
SET i=i+1;
```

```
END WHILE;
```

```
END //
```

```
DELIMITER ;
```

```
CALL generate_transactions();
SELECT COUNT(*) FROM customers; SELECT COUNT(*) FROM accounts; SELECT COUNT(*)
FROM transactions;
```

Q2. Perform a full Sqoop import of customers, accounts and transactions.

```
hdfs dfs -rm -r -f /me/bank
hdfs dfs -mkdir -p /me/bank
sqoop import --connect jdbc:mysql://localhost:3306/bank_db?useSSL=false --username msis --password
mahe --table customers --target-dir /me/bank/customers --num-mappers 1
sqoop import --connect jdbc:mysql://localhost:3306/bank_db?useSSL=false --username msis --password
mahe --table accounts --target-dir /me/bank/accounts --num-mappers 1
sqoop import --connect jdbc:mysql://localhost:3306/bank_db?useSSL=false --username msis --password
mahe --table transactions --target-dir /me/bank/transactions --num-mappers 1
```

Q3. Verify through HDFS that the new data has been imported.

```
hdfs dfs -ls /me/bank
hdfs dfs -cat /me/bank/customers/part-* | wc -l
hdfs dfs -cat /me/bank/accounts/part-* | wc -l
hdfs dfs -cat /me/bank/transactions/part-* | wc -l
```

Q4. Create Hive tables and answer the seven questions.

```
hive
CREATE DATABASE IF NOT EXISTS bank_dw;
USE bank_dw;
CREATE EXTERNAL TABLE customers(customer_id INT,customer_name STRING,city
STRING,account_type STRING,account_open_date DATE) ROW FORMAT DELIMITED FIELDS
TERMINATED BY ',' STORED AS TEXTFILE LOCATION '/me/bank/customers';
CREATE EXTERNAL TABLE accounts(account_id INT,customer_id INT,balance
DECIMAL(12,2),branch STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE LOCATION '/me/bank/accounts';
CREATE EXTERNAL TABLE transactions(transaction_id INT,account_id INT,transaction_type
STRING,amount DECIMAL(12,2),transaction_date DATE) ROW FORMAT DELIMITED FIELDS
TERMINATED BY ',' STORED AS TEXTFILE LOCATION '/me/bank/transactions';
```

Q4.1 Total deposits and withdrawals

```
SELECT transaction_type,SUM(amount) total_amount FROM transactions GROUP BY transaction_type;
```

Q4.2 Customers with the highest transaction values

```
SELECT c.customer_id,c.customer_name,SUM(t.amount) total_transaction_value FROM customers c
JOIN accounts a ON c.customer_id=a.customer_id JOIN transactions t ON a.account_id=t.account_id
GROUP BY c.customer_id,c.customer_name ORDER BY total_transaction_value DESC LIMIT 5;
```

Q4.3 Branch-wise account balances

```
SELECT branch,SUM(balance) total_balance FROM accounts GROUP BY branch ORDER BY
total_balance DESC;
```

Q4.4 Average transaction amount by account type

```
SELECT c.account_type,AVG(t.amount) average_transaction_amount FROM customers c JOIN accounts
a ON c.customer_id=a.customer_id JOIN transactions t ON a.account_id=t.account_id GROUP BY
c.account_type;
```

Q4.5 Month-wise transaction volume

```
SELECT YEAR(transaction_date) year,MONTH(transaction_date) month,COUNT(*)
transaction_volume FROM transactions GROUP BY YEAR(transaction_date),MONTH(transaction_date)
ORDER BY year,month;
```

Q4.6 Accounts with no transactions

```
SELECT a.account_id,a.customer_id,a.balance,a.branch FROM accounts a LEFT JOIN transactions t ON
a.account_id=t.account_id WHERE t.account_id IS NULL;
```

Q4.7 Customers whose withdrawal amount exceeds their deposit amount

```
SELECT c.customer_id,c.customer_name,SUM(CASE WHEN t.transaction_type='Withdrawal' THEN
t.amount ELSE 0 END) withdrawals,SUM(CASE WHEN t.transaction_type='Deposit' THEN t.amount
ELSE 0 END) deposits FROM customers c JOIN accounts a ON c.customer_id=a.customer_id JOIN
transactions t ON a.account_id=t.account_id GROUP BY c.customer_id,c.customer_name HAVING
withdrawals>deposits;
```

Q5. Store each result separately in HDFS.

```
exit
hdfs dfs -mkdir -p /me/bank/results
```

```
hive -S -e "USE bank_dw; SELECT transaction_type,SUM(amount) FROM transactions GROUP BY
transaction_type;" > /tmp/deposits_withdrawals.txt
hdfs dfs -put -f /tmp/deposits_withdrawals.txt /me/bank/results/
```

```
hive -S -e "USE bank_dw; SELECT c.customer_id,c.customer_name,SUM(t.amount) FROM customers c JOIN
accounts a ON c.customer_id=a.customer_id JOIN transactions t ON a.account_id=t.account_id GROUP BY
c.customer_id,c.customer_name ORDER BY SUM(t.amount) DESC LIMIT 5;" >
/tmp/top_customers_transactions.txt
hdfs dfs -put -f /tmp/top_customers_transactions.txt /me/bank/results/
```

```
hive -S -e "USE bank_dw; SELECT branch,SUM(balance) FROM accounts GROUP BY branch;" >
/tmp/branch_balances.txt
hdfs dfs -put -f /tmp/branch_balances.txt /me/bank/results/
```

```
hive -S -e "USE bank_dw; SELECT c.account_type,AVG(t.amount) FROM customers c JOIN accounts a ON
c.customer_id=a.customer_id JOIN transactions t ON a.account_id=t.account_id GROUP BY c.account_type;"
> /tmp/avg_transaction_account_type.txt
hdfs dfs -put -f /tmp/avg_transaction_account_type.txt /me/bank/results/
```

```
hive -S -e "USE bank_dw; SELECT YEAR(transaction_date),MONTH(transaction_date),COUNT(*) FROM
transactions GROUP BY YEAR(transaction_date),MONTH(transaction_date);" >
/tmp/month_transaction_volume.txt
hdfs dfs -put -f /tmp/month_transaction_volume.txt /me/bank/results/
```

```
hive -S -e "USE bank_dw; SELECT a.account_id,a.customer_id,a.balance,a.branch FROM accounts a LEFT
JOIN transactions t ON a.account_id=t.account_id WHERE t.account_id IS NULL;" >
/tmp/accounts_no_transactions.txt
hdfs dfs -put -f /tmp/accounts_no_transactions.txt /me/bank/results/
```

```
hive -S -e "USE bank_dw; SELECT c.customer_id,c.customer_name,SUM(CASE WHEN
t.transaction_type='Withdrawal' THEN t.amount ELSE 0 END),SUM(CASE WHEN
t.transaction_type='Deposit' THEN t.amount ELSE 0 END) FROM customers c JOIN accounts a ON
c.customer_id=a.customer_id JOIN transactions t ON a.account_id=t.account_id GROUP BY
c.customer_id,c.customer_name HAVING SUM(CASE WHEN t.transaction_type='Withdrawal' THEN
t.amount ELSE 0 END)>SUM(CASE WHEN t.transaction_type='Deposit' THEN t.amount ELSE 0 END);" >
/tmp/withdrawal_exceeds_deposit.txt
hdfs dfs -put -f /tmp/withdrawal_exceeds_deposit.txt /me/bank/results/
```

```
hdfs dfs -ls /me/bank/results
```

Q6. Transfer result files back to local system.

```
mkdir -p ~/bank_results
hdfs dfs -get /me/bank/results/* ~/bank_results/
ls ~/bank_results
```

ASSIGNMENT 4 — HOSPITAL OPERATIONS ANALYTICS

Scenario: A multi-specialty hospital wants to analyze patient visits, doctors, departments, and billing information stored in MySQL.

MySQL tables: patients(patient_id, patient_name, city, age, gender), doctors(doctor_id, doctor_name, specialization, department), visits(visit_id, patient_id, doctor_id, visit_date, diagnosis, consultation_fee), and payments(payment_id, visit_id, payment_mode, amount, payment_status)

Students should create at least 40 patients, 12 doctors, 150 visits, and 150 payment records using synthetic data only.

Q1. Create and populate hospital_db using synthetic data only: 40 patients, 12 doctors, 150 visits, 150 payments.

```
mysql -u msis -p
DROP DATABASE IF EXISTS hospital_db;
CREATE DATABASE hospital_db;
USE hospital_db;

CREATE TABLE patients(patient_id INT PRIMARY KEY,patient_name VARCHAR(100),city
VARCHAR(50),age INT,gender VARCHAR(10));
DROP PROCEDURE IF EXISTS generate_patients;
DELIMITER //
CREATE PROCEDURE generate_patients()
BEGIN
DECLARE i INT DEFAULT 1;
WHILE i<=40 DO
```

```

INSERT INTO patients VALUES(i,CONCAT('Patient',i),
CASE MOD(i,5) WHEN 0 THEN 'Bengaluru' WHEN 1 THEN 'Mysuru' WHEN 2 THEN 'Chennai'
WHEN 3 THEN 'Hyderabad' ELSE 'Pune' END,
18+MOD(i*3,63),CASE WHEN MOD(i,2)=0 THEN 'Female' ELSE 'Male' END);
SET i=i+1;
END WHILE;
END //
DELIMITER ;
CALL generate_patients();

```

```

CREATE TABLE doctors(doctor_id INT PRIMARY KEY,doctor_name VARCHAR(100),specialization
VARCHAR(100),department VARCHAR(100));
INSERT INTO doctors VALUES
(1,'Dr. Arun','Cardiology','Cardiology'),(2,'Dr. Priya','Neurology','Neurology'),
(3,'Dr. Ravi','Orthopedics','Orthopedics'),(4,'Dr. Sneha','Dermatology','Dermatology'),
(5,'Dr. Kiran','Pediatrics','Pediatrics'),(6,'Dr. Anjali','General Medicine','General Medicine'),
(7,'Dr. Mohan','Cardiology','Cardiology'),(8,'Dr. Divya','Neurology','Neurology'),
(9,'Dr. Suresh','Orthopedics','Orthopedics'),(10,'Dr. Neha','Dermatology','Dermatology'),
(11,'Dr. Varun','Pediatrics','Pediatrics'),(12,'Dr. Kavya','General Medicine','General Medicine');

```

```

CREATE TABLE visits(visit_id INT PRIMARY KEY,patient_id INT,doctor_id INT,visit_date
DATE,diagnosis VARCHAR(100),consultation_fee DECIMAL(10,2),FOREIGN KEY(patient_id)
REFERENCES patients(patient_id),FOREIGN KEY(doctor_id) REFERENCES doctors(doctor_id));
DROP PROCEDURE IF EXISTS generate_visits;
DELIMITER //
CREATE PROCEDURE generate_visits()
BEGIN
DECLARE i INT DEFAULT 1;
WHILE i<=150 DO
INSERT INTO visits VALUES(i,MOD(i-1,40)+1,MOD(i-1,12)+1,DATE_ADD('2024-01-
01',INTERVAL MOD(i-1,365) DAY),
CASE MOD(i,6) WHEN 0 THEN 'Fever' WHEN 1 THEN 'Diabetes' WHEN 2 THEN 'Hypertension'
WHEN 3 THEN 'Migraine' WHEN 4 THEN 'Fracture' ELSE 'Skin Allergy' END,
500+MOD(i*75,1500));
SET i=i+1;
END WHILE;
END //
DELIMITER ;
CALL generate_visits();

```

```

CREATE TABLE payments(payment_id INT PRIMARY KEY,visit_id INT,payment_mode
VARCHAR(30),amount DECIMAL(10,2),payment_status VARCHAR(30),FOREIGN KEY(visit_id)
REFERENCES visits(visit_id));
DROP PROCEDURE IF EXISTS generate_payments;
DELIMITER //
CREATE PROCEDURE generate_payments()
BEGIN
DECLARE i INT DEFAULT 1;
WHILE i<=150 DO
INSERT INTO payments VALUES(i,i,CASE MOD(i,3) WHEN 0 THEN 'Cash' WHEN 1 THEN 'Card'
ELSE 'UPI' END,500+MOD(i*75,1500),CASE WHEN MOD(i,5)=0 THEN 'Pending' ELSE 'Successful'

```

```

END);
SET i=i+1;
END WHILE;
END //
DELIMITER ;
CALL generate_payments();

SELECT COUNT(*) FROM patients; SELECT COUNT(*) FROM doctors;
SELECT COUNT(*) FROM visits; SELECT COUNT(*) FROM payments;

```

Q2. Import the four tables using Sqoop.

```

hdfs dfs -rm -r -f /me/hospital
hdfs dfs -mkdir -p /me/hospital
sqoop import --connect jdbc:mysql://localhost:3306/hospital_db?useSSL=false --username msis --
password mahe --table patients --target-dir /me/hospital/patients --num-mappers 1
sqoop import --connect jdbc:mysql://localhost:3306/hospital_db?useSSL=false --username msis --
password mahe --table doctors --target-dir /me/hospital/doctors --num-mappers 1
sqoop import --connect jdbc:mysql://localhost:3306/hospital_db?useSSL=false --username msis --
password mahe --table visits --target-dir /me/hospital/visits --num-mappers 1
sqoop import --connect jdbc:mysql://localhost:3306/hospital_db?useSSL=false --username msis --
password mahe --table payments --target-dir /me/hospital/payments --num-mappers 1

```

Q3. Store them under a structured HDFS hierarchy.

```

hdfs dfs -ls /me/hospital
hdfs dfs -ls /me/hospital/patients
hdfs dfs -ls /me/hospital/doctors
hdfs dfs -ls /me/hospital/visits
hdfs dfs -ls /me/hospital/payments

```

Q4. Create Hive tables.

```

hive
CREATE DATABASE IF NOT EXISTS hospital_dw;
USE hospital_dw;
CREATE EXTERNAL TABLE patients(patient_id INT,patient_name STRING,city STRING,age
INT,gender STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS
TEXTFILE LOCATION '/me/hospital/patients';
CREATE EXTERNAL TABLE doctors(doctor_id INT,doctor_name STRING,specialization
STRING,department STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED
AS TEXTFILE LOCATION '/me/hospital/doctors';
CREATE EXTERNAL TABLE visits(visit_id INT,patient_id INT,doctor_id INT,visit_date
DATE,diagnosis STRING,consultation_fee DECIMAL(10,2)) ROW FORMAT DELIMITED FIELDS
TERMINATED BY ',' STORED AS TEXTFILE LOCATION '/me/hospital/visits';
CREATE EXTERNAL TABLE payments(payment_id INT,visit_id INT,payment_mode STRING,amount
DECIMAL(10,2),payment_status STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY
',' STORED AS TEXTFILE LOCATION '/me/hospital/payments';

```

Q5. Use Hive joins and aggregations to determine:

Q5.1 Number of visits per department

```
SELECT d.department,COUNT(v.visit_id) number_of_visits FROM doctors d JOIN visits v ON
d.doctor_id=v.doctor_id GROUP BY d.department ORDER BY number_of_visits DESC;
```

Q5.2 Revenue generated by each department

```
SELECT d.department,SUM(v.consultation_fee) revenue FROM doctors d JOIN visits v ON
d.doctor_id=v.doctor_id GROUP BY d.department ORDER BY revenue DESC;
```

Q5.3 Top 5 doctors based on number of consultations

```
SELECT d.doctor_id,d.doctor_name,COUNT(v.visit_id) consultations FROM doctors d JOIN visits v ON
d.doctor_id=v.doctor_id GROUP BY d.doctor_id,d.doctor_name ORDER BY consultations DESC
LIMIT 5;
```

Q5.4 Average consultation fee by specialization

```
SELECT d.specialization,AVG(v.consultation_fee) average_fee FROM doctors d JOIN visits v ON
d.doctor_id=v.doctor_id GROUP BY d.specialization;
```

Q5.5 Most common diagnoses

```
SELECT diagnosis,COUNT(*) occurrence FROM visits GROUP BY diagnosis ORDER BY occurrence
DESC;
```

Q5.6 City-wise patient counts

```
SELECT city,COUNT(*) patient_count FROM patients GROUP BY city ORDER BY patient_count
DESC;
```

Q5.7 Percentage of successful versus pending payments

```
SELECT payment_status,COUNT(*) count,ROUND(100.0*COUNT(*)/(SELECT COUNT(*) FROM
payments),2) percentage FROM payments GROUP BY payment_status;
```

Q5.8 Month having the highest number of visits

```
SELECT YEAR(visit_date) year,MONTH(visit_date) month,COUNT(*) visit_count FROM visits
GROUP BY YEAR(visit_date),MONTH(visit_date) ORDER BY visit_count DESC LIMIT 1;
```

Q6. Store each result separately in HDFS.

```
exit
hdfs dfs -mkdir -p /me/hospital/results
```

```
hive -S -e "USE hospital_dw; SELECT d.department,COUNT(v.visit_id) FROM doctors d JOIN visits v ON
d.doctor_id=v.doctor_id GROUP BY d.department;" > /tmp/visits_department.txt
hdfs dfs -put -f /tmp/visits_department.txt /me/hospital/results/
```

```
hive -S -e "USE hospital_dw; SELECT d.department,SUM(v.consultation_fee) FROM doctors d JOIN visits v
ON d.doctor_id=v.doctor_id GROUP BY d.department;" > /tmp/revenue_department.txt
hdfs dfs -put -f /tmp/revenue_department.txt /me/hospital/results/
```

```
hive -S -e "USE hospital_dw; SELECT d.doctor_id,d.doctor_name,COUNT(v.visit_id) FROM doctors d JOIN visits v ON d.doctor_id=v.doctor_id GROUP BY d.doctor_id,d.doctor_name ORDER BY COUNT(v.visit_id) DESC LIMIT 5;" > /tmp/top5_doctors.txt
```

```
hdfs dfs -put -f /tmp/top5_doctors.txt /me/hospital/results/
```

```
hive -S -e "USE hospital_dw; SELECT d.specialization,AVG(v.consultation_fee) FROM doctors d JOIN visits v ON d.doctor_id=v.doctor_id GROUP BY d.specialization;" > /tmp/avg_fee_specialization.txt
```

```
hdfs dfs -put -f /tmp/avg_fee_specialization.txt /me/hospital/results/
```

```
hive -S -e "USE hospital_dw; SELECT diagnosis,COUNT(*) FROM visits GROUP BY diagnosis ORDER BY COUNT(*) DESC;" > /tmp/common_diagnoses.txt
```

```
hdfs dfs -put -f /tmp/common_diagnoses.txt /me/hospital/results/
```

```
hive -S -e "USE hospital_dw; SELECT city,COUNT(*) FROM patients GROUP BY city;" > /tmp/city_patient_count.txt
```

```
hdfs dfs -put -f /tmp/city_patient_count.txt /me/hospital/results/
```

```
hive -S -e "USE hospital_dw; SELECT payment_status,COUNT(*),ROUND(100.0*COUNT(*)/(SELECT COUNT(*) FROM payments),2) FROM payments GROUP BY payment_status;" > /tmp/payment_percentage.txt
```

```
hdfs dfs -put -f /tmp/payment_percentage.txt /me/hospital/results/
```

```
hive -S -e "USE hospital_dw; SELECT YEAR(visit_date),MONTH(visit_date),COUNT(*) FROM visits GROUP BY YEAR(visit_date),MONTH(visit_date) ORDER BY COUNT(*) DESC LIMIT 1;" > /tmp/highest_visit_month.txt
```

```
hdfs dfs -put -f /tmp/highest_visit_month.txt /me/hospital/results/
```

```
hdfs dfs -ls /me/hospital/results
```

Q7. Transfer result files back to local system.

```
mkdir -p ~/hospital_results
```

```
hdfs dfs -get /me/hospital/results/* ~/hospital_results/
```

```
ls ~/hospital_results
```

ASSIGNMENT 5 — RETAIL DATA WAREHOUSE — COMPLETE ETL CHALLENGE

Scenario: A retail chain has multiple stores. Operational information is maintained in MySQL, while management wants a Hadoop/Hive analytical warehouse. This assignment combines the concepts from the previous four and is the most advanced.

MySQL tables: stores(store_id, store_name, city, state), customers(customer_id, customer_name, city, customer_segment), products(product_id, product_name, category, unit_price), and sales(sale_id, store_id, customer_id, product_id, quantity, discount, sale_date).

Q1. Design and create retail_data in MySQL with appropriate primary and foreign keys.

```
mysql -u msis -p
DROP DATABASE IF EXISTS retail_data;
CREATE DATABASE retail_data;
USE retail_data;

CREATE TABLE stores(store_id INT PRIMARY KEY,store_name VARCHAR(100),city
VARCHAR(50),state VARCHAR(50));
INSERT INTO stores VALUES
(1,'Store 1','Bengaluru','Karnataka'),(2,'Store 2','Mysuru','Karnataka'),
(3,'Store 3','Chennai','Tamil Nadu'),(4,'Store 4','Hyderabad','Telangana'),
(5,'Store 5','Pune','Maharashtra');

CREATE TABLE customers(customer_id INT PRIMARY KEY,customer_name VARCHAR(100),city
VARCHAR(50),customer_segment VARCHAR(50));
DROP PROCEDURE IF EXISTS generate_retail_customers;
DELIMITER //
CREATE PROCEDURE generate_retail_customers()
BEGIN
DECLARE i INT DEFAULT 1;
WHILE i<=50 DO
INSERT INTO customers VALUES(i,CONCAT('Customer',i),
CASE MOD(i,5) WHEN 0 THEN 'Bengaluru' WHEN 1 THEN 'Mysuru' WHEN 2 THEN 'Chennai'
WHEN 3 THEN 'Hyderabad' ELSE 'Pune' END,
CASE MOD(i,3) WHEN 0 THEN 'Premium' WHEN 1 THEN 'Regular' ELSE 'Occasional' END);
SET i=i+1;
END WHILE;
END //
DELIMITER ;
CALL generate_retail_customers();

CREATE TABLE products(product_id INT PRIMARY KEY,product_name VARCHAR(100),category
VARCHAR(50),unit_price DECIMAL(10,2));
INSERT INTO products VALUES
(1,'Laptop','Electronics',55000),(2,'Mobile','Electronics',25000),
(3,'Headphones','Electronics',2000),(4,'Keyboard','Electronics',1200),
(5,'Mouse','Electronics',800),(6,'Monitor','Electronics',12000),
(7,'T-Shirt','Fashion',900),(8,'Jeans','Fashion',1800),
(9,'Shoes','Fashion',2500),(10,'Jacket','Fashion',3000),
(11,'Backpack','Fashion',1500),(12,'Book','Books',500),
(13,'Notebook','Books',200),(14,'Pen','Books',50),
(15,'Calculator','Books',700),(16,'Chair','Furniture',5000),
(17,'Table','Furniture',8000),(18,'Lamp','Home',1800),
(19,'Mixer','Home',4500),(20,'Air Cooler','Home',10000);

CREATE TABLE sales(sale_id INT PRIMARY KEY,store_id INT,customer_id INT,product_id INT,
quantity INT,discount DECIMAL(5,2),sale_date DATE,
FOREIGN KEY(store_id) REFERENCES stores(store_id),
FOREIGN KEY(customer_id) REFERENCES customers(customer_id),
FOREIGN KEY(product_id) REFERENCES products(product_id));
```

Q2. Populate realistic synthetic data, including some NULL values and discounted transactions.

```

DROP PROCEDURE IF EXISTS generate_sales;
DELIMITER //
CREATE PROCEDURE generate_sales()
BEGIN
DECLARE i INT DEFAULT 1;
WHILE i<=250 DO
INSERT INTO sales VALUES(i,MOD(i-1,5)+1,MOD(i-1,50)+1,MOD(i-1,19)+1,
MOD(i,5)+1,
CASE WHEN MOD(i,5)=0 THEN 20 WHEN MOD(i,3)=0 THEN 10 ELSE 0 END,
DATE_ADD('2024-01-01',INTERVAL MOD(i-1,365) DAY));
SET i=i+1;
END WHILE;
END //
DELIMITER ;
CALL generate_sales();

UPDATE customers SET city=NULL WHERE customer_id IN (10,20,30);
UPDATE products SET unit_price=NULL WHERE product_id=19;

SELECT COUNT(*) FROM stores;
SELECT COUNT(*) FROM customers;
SELECT COUNT(*) FROM products;
SELECT COUNT(*) FROM sales;
SELECT discount,COUNT(*) FROM sales GROUP BY discount;

```

Product 20 is intentionally not used in sales, so Q8.9 (Products never sold) produces a meaningful result.
Product 19 has a NULL price to demonstrate cleaning.

Q3. Import dimension tables using Sqoop.

```

hdfs dfs -rm -r -f /me/retail
hdfs dfs -mkdir -p /me/retail

sqoop import --connect jdbc:mysql://localhost:3306/retail_data?useSSL=false --username msis --
password mahe --table stores --target-dir /me/retail/stores --num-mappers 1
sqoop import --connect jdbc:mysql://localhost:3306/retail_data?useSSL=false --username msis --
password mahe --table customers --target-dir /me/retail/customers --num-mappers 1
sqoop import --connect jdbc:mysql://localhost:3306/retail_data?useSSL=false --username msis --
password mahe --table products --target-dir /me/retail/products --num-mappers 1

# Import the sales fact table as it is required for the Hive ETL and analytics.
sqoop import --connect jdbc:mysql://localhost:3306/retail_data?useSSL=false --username msis --
password mahe --table sales --target-dir /me/retail/sales --num-mappers 1

```

Q4. Store the imported datasets in an organized HDFS directory structure.

```

hdfs dfs -ls /me/retail
hdfs dfs -ls /me/retail/stores
hdfs dfs -ls /me/retail/customers

```

```
hdfs dfs -ls /me/retail/products
hdfs dfs -ls /me/retail/sales
hdfs dfs -cat /me/retail/stores/part-* | wc -l
hdfs dfs -cat /me/retail/customers/part-* | wc -l
hdfs dfs -cat /me/retail/products/part-* | wc -l
hdfs dfs -cat /me/retail/sales/part-* | wc -l
```

Q5. Create Hive database retail_dw and tables.

```
hive
CREATE DATABASE IF NOT EXISTS retail_dw;
USE retail_dw;

CREATE EXTERNAL TABLE stores(store_id INT,store_name STRING,city STRING,state STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION
'/me/retail/stores';

CREATE EXTERNAL TABLE customers(customer_id INT,customer_name STRING,city
STRING,customer_segment STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION
'/me/retail/customers';

CREATE EXTERNAL TABLE products(product_id INT,product_name STRING,category
STRING,unit_price DECIMAL(10,2))
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION
'/me/retail/products';

CREATE EXTERNAL TABLE sales(sale_id INT,store_id INT,customer_id INT,product_id
INT,quantity INT,discount DECIMAL(5,2),sale_date DATE)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION
'/me/retail/sales';
```

Q6. Perform data cleaning/transformation in Hive, including NULL handling and calculations.

```
CREATE TABLE sales_cleaned AS
SELECT s.sale_id,s.store_id,s.customer_id,s.product_id,s.quantity,
COALESCE(p.unit_price,0) unit_price,
COALESCE(s.discount,0) discount,
s.sale_date,
s.quantity*COALESCE(p.unit_price,0) gross_amount,
(s.quantity*COALESCE(p.unit_price,0))*(1-COALESCE(s.discount,0)/100) net_amount
FROM sales s LEFT JOIN products p ON s.product_id=p.product_id;

SELECT * FROM sales_cleaned LIMIT 10;
```

Required formulas: $\text{gross_amount} = \text{quantity} \times \text{unit_price}$; $\text{net_amount} = \text{gross_amount} \times (1 - \text{discount}/100)$.
COALESCE handles NULLs.

Q8. Write analytical queries for:

Q8.1 Store-wise revenue

```
SELECT st.store_id,st.store_name,SUM(sc.net_amount) revenue FROM sales_cleaned sc JOIN stores st
ON sc.store_id=st.store_id GROUP BY st.store_id,st.store_name ORDER BY revenue DESC;
```

Q8.2 State-wise revenue

```
SELECT st.state,SUM(sc.net_amount) revenue FROM sales_cleaned sc JOIN stores st ON
sc.store_id=st.store_id GROUP BY st.state ORDER BY revenue DESC;
```

Q8.3 Category-wise revenue

```
SELECT p.category,SUM(sc.net_amount) revenue FROM sales_cleaned sc JOIN products p ON
sc.product_id=p.product_id GROUP BY p.category ORDER BY revenue DESC;
```

Q8.4 Top 10 customers

```
SELECT c.customer_id,c.customer_name,SUM(sc.net_amount) total_spending FROM sales_cleaned sc
JOIN customers c ON sc.customer_id=c.customer_id GROUP BY c.customer_id,c.customer_name
ORDER BY total_spending DESC LIMIT 10;
```

Q8.5 Top 5 products

```
SELECT p.product_id,p.product_name,SUM(sc.net_amount) revenue FROM sales_cleaned sc JOIN
products p ON sc.product_id=p.product_id GROUP BY p.product_id,p.product_name ORDER BY
revenue DESC LIMIT 5;
```

Q8.6 Monthly sales trends

```
SELECT YEAR(sale_date) year,MONTH(sale_date) month,SUM(net_amount) monthly_revenue FROM
sales_cleaned GROUP BY YEAR(sale_date),MONTH(sale_date) ORDER BY year,month;
```

Q8.7 Average order value by customer segment

```
SELECT c.customer_segment,AVG(sc.net_amount) average_order_value FROM sales_cleaned sc JOIN
customers c ON sc.customer_id=c.customer_id GROUP BY c.customer_segment;
```

Q8.8 Store contributing the highest percentage of total revenue

```
SELECT st.store_name,SUM(sc.net_amount)
store_revenue,ROUND(100.0*SUM(sc.net_amount)/(SELECT SUM(net_amount) FROM
sales_cleaned),2) revenue_percentage FROM sales_cleaned sc JOIN stores st ON sc.store_id=st.store_id
GROUP BY st.store_name ORDER BY revenue_percentage DESC LIMIT 1;
```

Q8.9 Products never sold

```
SELECT p.product_id,p.product_name,p.category FROM products p LEFT JOIN sales s ON
p.product_id=s.product_id WHERE s.product_id IS NULL;
```

Q8.10 Month-over-month revenue growth

```
WITH monthly_sales AS (SELECT YEAR(sale_date) year,MONTH(sale_date) month,SUM(net_amount)
revenue FROM sales_cleaned GROUP BY YEAR(sale_date),MONTH(sale_date)), previous_month AS
(SELECT year,month,revenue,LAG(revenue) OVER(ORDER BY year,month) previous_revenue FROM
monthly_sales) SELECT year,month,revenue,previous_revenue,ROUND(100.0*(revenue-
```

```
previous_revenue)/previous_revenue,2) growth_percentage FROM previous_month ORDER BY
year,month;
```

Q8 continued — Store each analytical result separately in HDFS.

```
exit
hdfs dfs -mkdir -p /me/retail/results
```

```
hive -S -e "USE retail_dw; SELECT st.store_id,st.store_name,SUM(sc.net_amount) FROM sales_cleaned sc
JOIN stores st ON sc.store_id=st.store_id GROUP BY st.store_id,st.store_name;" > /tmp/store_revenue.txt
hdfs dfs -put -f /tmp/store_revenue.txt /me/retail/results/
```

```
hive -S -e "USE retail_dw; SELECT st.state,SUM(sc.net_amount) FROM sales_cleaned sc JOIN stores st ON
sc.store_id=st.store_id GROUP BY st.state;" > /tmp/state_revenue.txt
hdfs dfs -put -f /tmp/state_revenue.txt /me/retail/results/
```

```
hive -S -e "USE retail_dw; SELECT p.category,SUM(sc.net_amount) FROM sales_cleaned sc JOIN products p
ON sc.product_id=p.product_id GROUP BY p.category;" > /tmp/category_revenue.txt
hdfs dfs -put -f /tmp/category_revenue.txt /me/retail/results/
```

```
hive -S -e "USE retail_dw; SELECT c.customer_id,c.customer_name,SUM(sc.net_amount) FROM
sales_cleaned sc JOIN customers c ON sc.customer_id=c.customer_id GROUP BY
c.customer_id,c.customer_name ORDER BY SUM(sc.net_amount) DESC LIMIT 10;" >
/tmp/top10_customers.txt
hdfs dfs -put -f /tmp/top10_customers.txt /me/retail/results/
```

```
hive -S -e "USE retail_dw; SELECT p.product_id,p.product_name,SUM(sc.net_amount) FROM sales_cleaned
sc JOIN products p ON sc.product_id=p.product_id GROUP BY p.product_id,p.product_name ORDER BY
SUM(sc.net_amount) DESC LIMIT 5;" > /tmp/top5_products.txt
hdfs dfs -put -f /tmp/top5_products.txt /me/retail/results/
```

```
hive -S -e "USE retail_dw; SELECT YEAR(sale_date),MONTH(sale_date),SUM(net_amount) FROM
sales_cleaned GROUP BY YEAR(sale_date),MONTH(sale_date) ORDER BY
YEAR(sale_date),MONTH(sale_date);" > /tmp/monthly_sales_trends.txt
hdfs dfs -put -f /tmp/monthly_sales_trends.txt /me/retail/results/
```

```
hive -S -e "USE retail_dw; SELECT c.customer_segment,AVG(sc.net_amount) FROM sales_cleaned sc JOIN
customers c ON sc.customer_id=c.customer_id GROUP BY c.customer_segment;" >
/tmp/avg_order_segment.txt
hdfs dfs -put -f /tmp/avg_order_segment.txt /me/retail/results/
```

```
hive -S -e "USE retail_dw; SELECT
st.store_name,SUM(sc.net_amount),100.0*SUM(sc.net_amount)/(SELECT SUM(net_amount) FROM
sales_cleaned) FROM sales_cleaned sc JOIN stores st ON sc.store_id=st.store_id GROUP BY st.store_name
ORDER BY 100.0*SUM(sc.net_amount)/(SELECT SUM(net_amount) FROM sales_cleaned) DESC LIMIT
1;" > /tmp/highest_store_percentage.txt
hdfs dfs -put -f /tmp/highest_store_percentage.txt /me/retail/results/
```

```
hive -S -e "USE retail_dw; SELECT p.product_id,p.product_name,p.category FROM products p LEFT JOIN
sales s ON p.product_id=s.product_id WHERE s.product_id IS NULL;" > /tmp/products_never_sold.txt
hdfs dfs -put -f /tmp/products_never_sold.txt /me/retail/results/
```

```
hive -S -e "USE retail_dw; WITH monthly_sales AS (SELECT YEAR(sale_date) year,MONTH(sale_date)
month,SUM(net_amount) revenue FROM sales_cleaned GROUP BY YEAR(sale_date),MONTH(sale_date)),
previous_month AS (SELECT year,month,revenue,LAG(revenue) OVER(ORDER BY year,month)
previous_revenue FROM monthly_sales) SELECT
year,month,revenue,previous_revenue,ROUND(100.0*(revenue-previous_revenue)/previous_revenue,2)
FROM previous_month;" > /tmp/month_over_month_growth.txt
hdfs dfs -put -f /tmp/month_over_month_growth.txt /me/retail/results/
```

```
hdfs dfs -ls /me/retail/results
```

Final transfer of Q8 result files to local system.

```
mkdir -p ~/retail_results
hdfs dfs -get /me/retail/results/* ~/retail_results/
ls ~/retail_results
```

FINAL VERIFICATION / SUBMISSION CHECKLIST

Hadoop/YARN

```
jps
```

```
hdfs dfs -ls /
```

```
yarn node -list
```

Sqoop connection

```
sqoop list-databases --connect jdbc:mysql://localhost?useSSL=false --username msis --password mahe
```

Assignment roots

```
hdfs dfs -ls /me/ecommerce
```

```
hdfs dfs -ls /me/university
```

```
hdfs dfs -ls /me/bank
```

```
hdfs dfs -ls /me/hospital
```

```
hdfs dfs -ls /me/retail
```

Local result folders

```
ls ~/ecommerce_results
```

```
ls ~/university_results
```

```
ls ~/bank_results
```

```
ls ~/hospital_results
```

```
ls ~/retail_results
```